CSCI 677 Homework

Hardware. Apple M1 Pro (16 GB RAM)

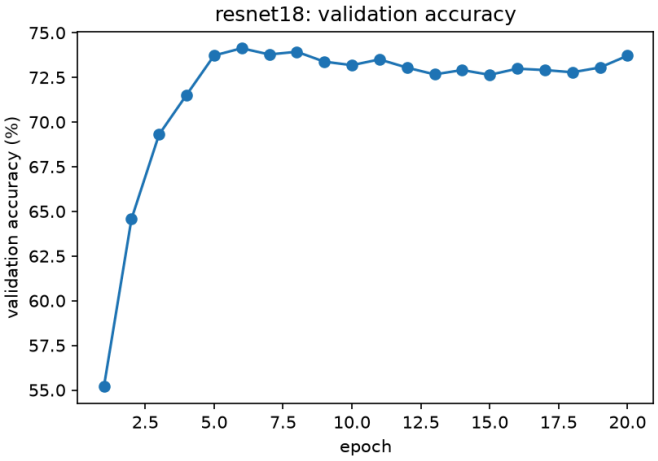
	ResNet-18	ViT full fine-tune	ViT frozen-head
Initialization	from scratch	google/vit-base-patch16-224-in21k	same
Trainable params	11,173,962	85,806,346	7,690 (head only)
Input size	32x32	224x224 (upsampled)	224x224 (upsampled)
Epochs	20	2	2
Batch size	32	16	16
Optimizer	Adam (β_1=0.9, β_2=0.999)	Adam	Adam
Learning rate	1e-3	5e-5	1e-3
Loss	cross-entropy	cross-entropy	cross-entropy
Training time	67 min	179 min	90 min

ResNet-18 architecture. Implemented layer by layer based on Figure 1 of the assignment

2. Training curves and validation accuracy

ResNet-18



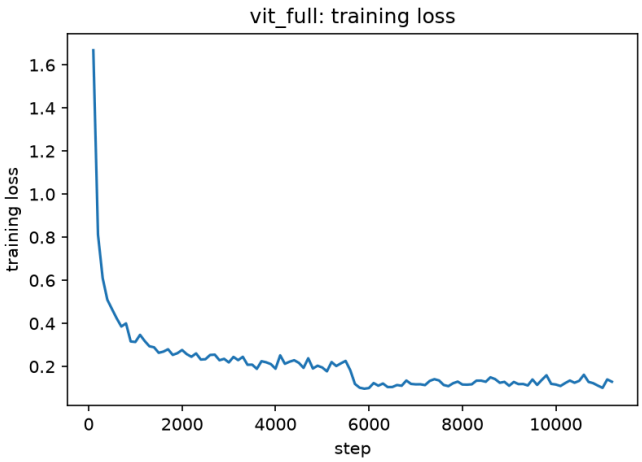


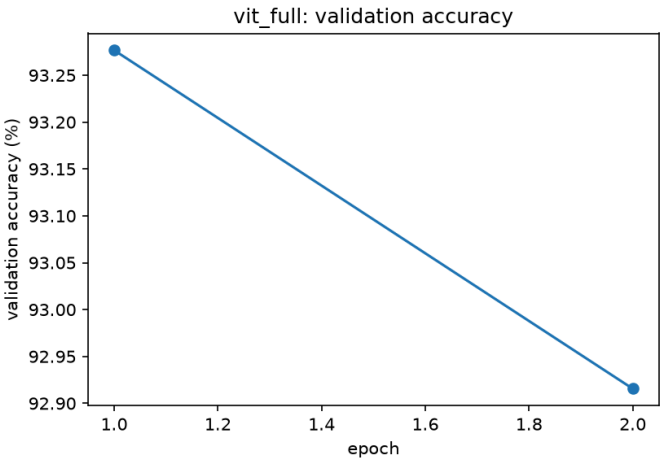
Validation accuracy by epoch (best 74.13% @ epoch 6):

Epoch	1	2	3	4	5	6	10	15	20
Val acc (%)	55.19	64.57	69.30	71.52	73.73	74.13	73.18	72.64	73.71

Validation accuracy rises quickly over the first five epochs and peaks at 74.13% in epoch 6. From then on it stays flat/ drifts down slightly while the training loss keeps falling. This is classic overfitting. The network is memorizing the training set rather than learning features that generalize. The test results below use the epoch-6 weights.

ViT (full fine-tune)

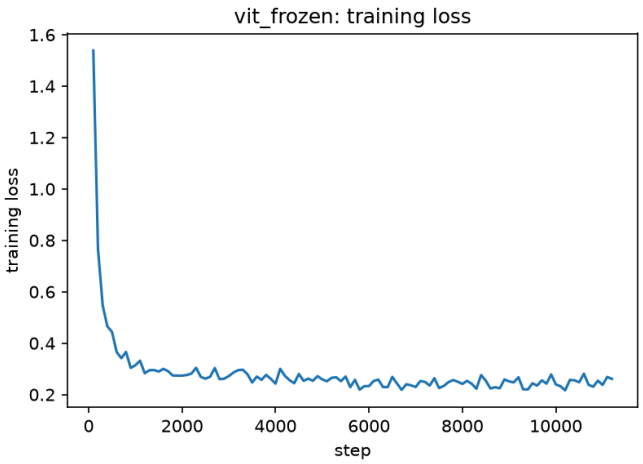


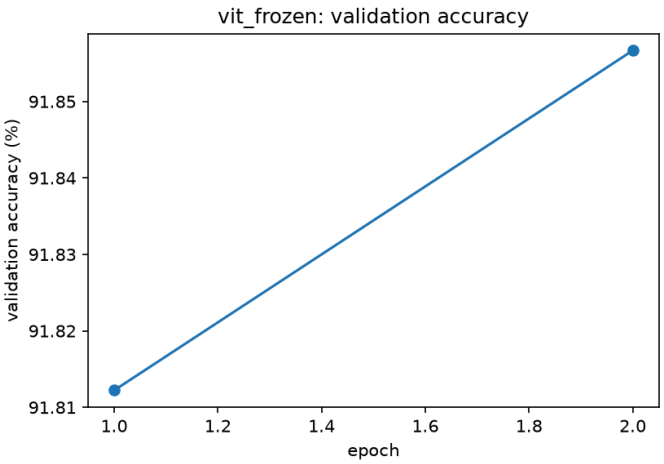


Epoch	1	2
Val acc (%)	93.28	92.92

The pretrained ViT already reaches 93.28% validation accuracy after one epoch which is already much better than the ResNet. In epoch 2 the mean training loss decreases significantly but validation accuracy drops. This suggests 1 epoch of training was sufficient. The epoch-1 checkpoint is used for testing.

ViT (frozen body, head only)





Epoch	1	2
Val acc (%)	91.81	91.86

The model reaches 91.81% after one epoch and gains just 0.05 points in the second, while the mean training loss falls more slowly than in full fine-tuning (0.32 -> 0.24). A linear classifier on fixed features has little capacity to overfit, so training is stable.

3. Test results

Overall top-1 test accuracy

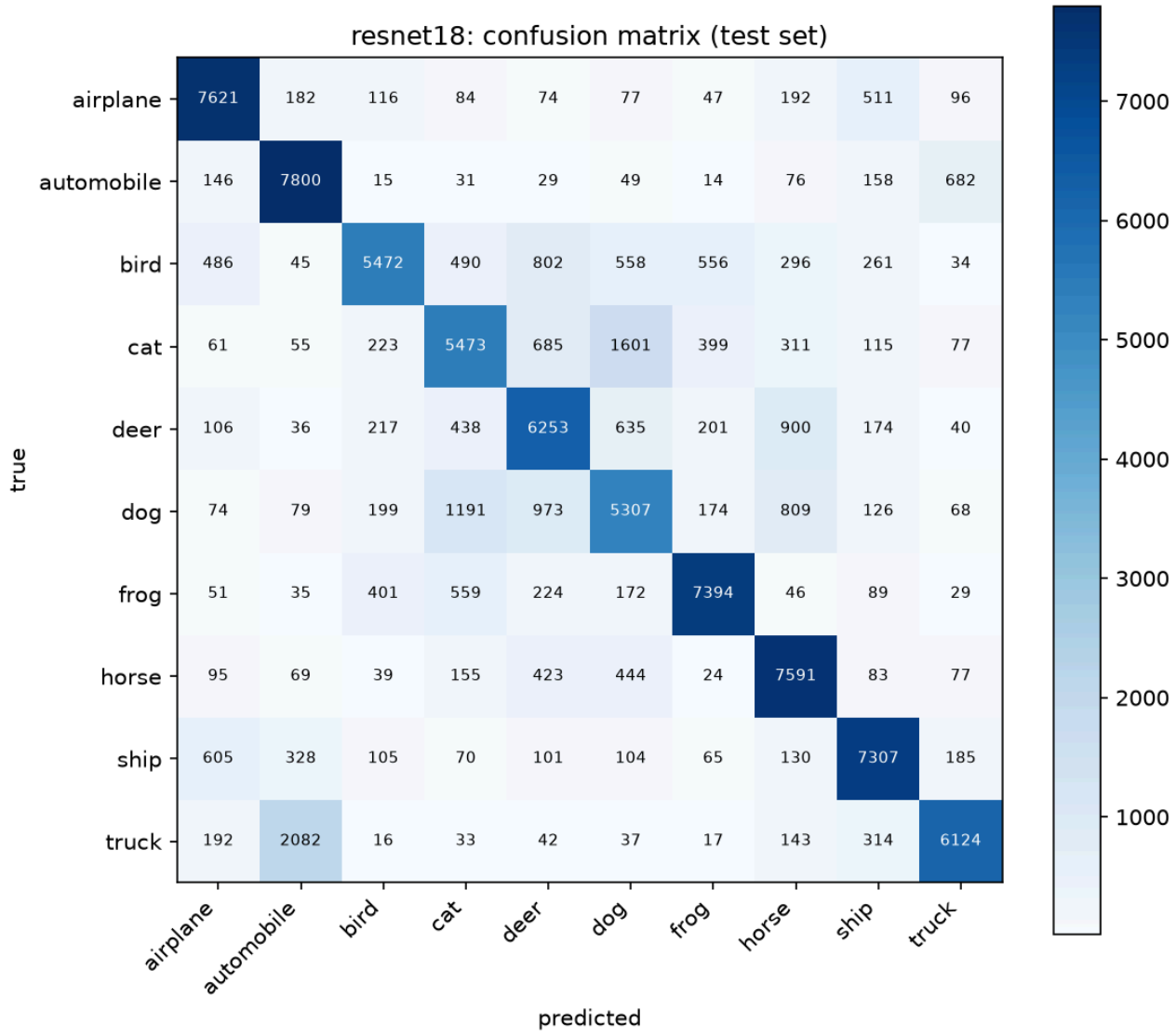
Model	Test accuracy
ResNet-18 (scratch)	73.71%

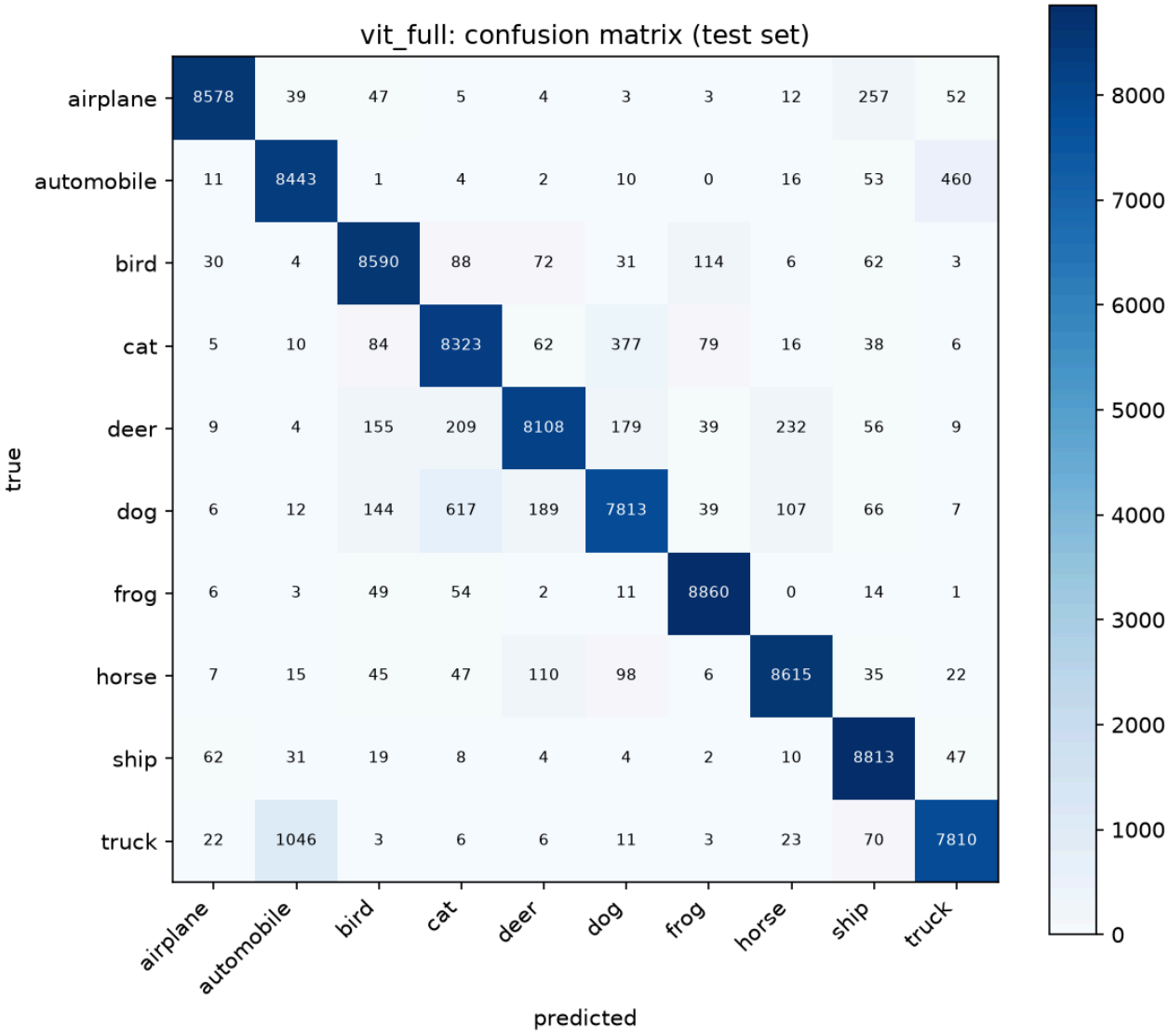
Model	Test accuracy
ViT full fine-tune	93.28%
ViT frozen-head	91.73%

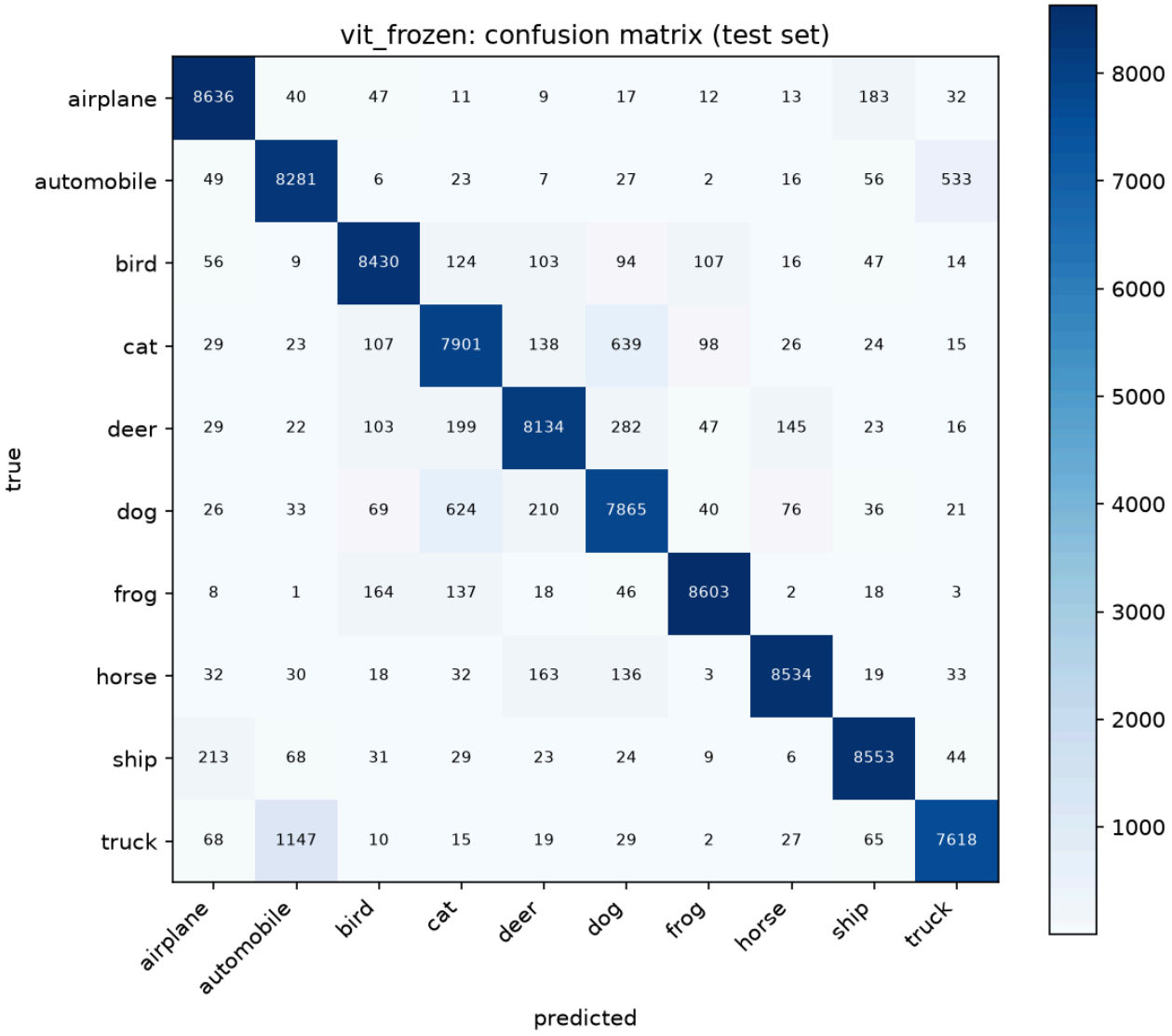
Per-class accuracy

Class	ResNet-18	ViT full	ViT frozen
airplane	84.68	95.31	95.96
automobile	86.67	93.81	92.01
bird	60.80	95.44	93.67
cat	60.81	92.48	87.79
deer	69.48	90.09	90.38
dog	58.97	86.81	87.39
frog	82.16	98.44	95.59
horse	84.34	95.72	94.82
ship	81.19	97.92	95.03
truck	68.04	86.78	84.64

Confusion matrices




vit_full: confusion matrix (test set)
true
predicted
airplane
automobile
bird
cat
deer
dog
frog
horse
ship
truck
8578 39 47 5 4 3 3 12 257 52
11 8443 1 4 2 10 0 16 53 460
30 4 8590 88 72 31 114 6 62 3
5 10 84 8323 62 377 79 16 38 6
9 4 155 209 8108 179 39 232 56 9
6 12 144 617 189 7813 39 107 66 7
6 3 49 54 2 11 8860 0 14 1
7 15 45 47 110 98 6 8615 35 22
62 31 19 8 4 4 2 10 8813 47
22 1046 3 6 6 11 3 23 70 7810
8000
7000
6000
5000
4000
3000
2000
1000
0



Largest off-diagonal confusions (true->predicted, counts out of 9,000/class):

- ResNet-18: truck->automobile (2,082), cat->dog (1,601), dog->cat (1,191), dog->deer (973), deer->horse (900)
- ViT full: truck->automobile (1,046), dog->cat (617), automobile->truck (460), cat->dog (377), airplane->ship (257)
- ViT frozen: truck->automobile (1,147), cat->dog (639), dog->cat (624), automobile->truck (533), deer->dog (282)

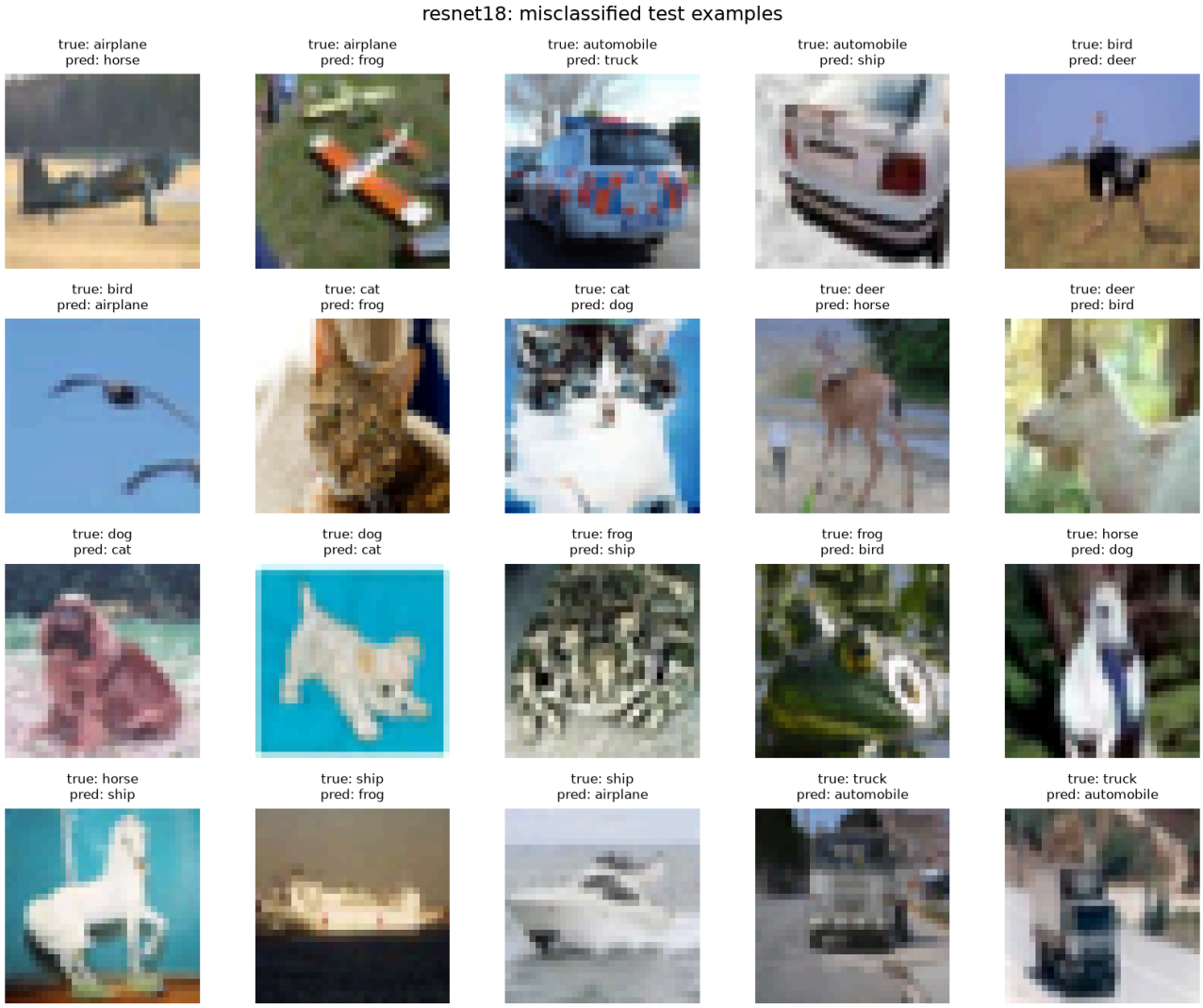
The same two semantically close pairs dominate all three confusion matrices: automobile<->truck and cat<->dog. Truck->automobile is the single largest error for every model. For ResNet-18, 60% of all errors are one animal mistaken for another and 23% are one vehicle mistaken for another; only 17% cross between animals and vehicles.

Pretraining reduces every kind of error:

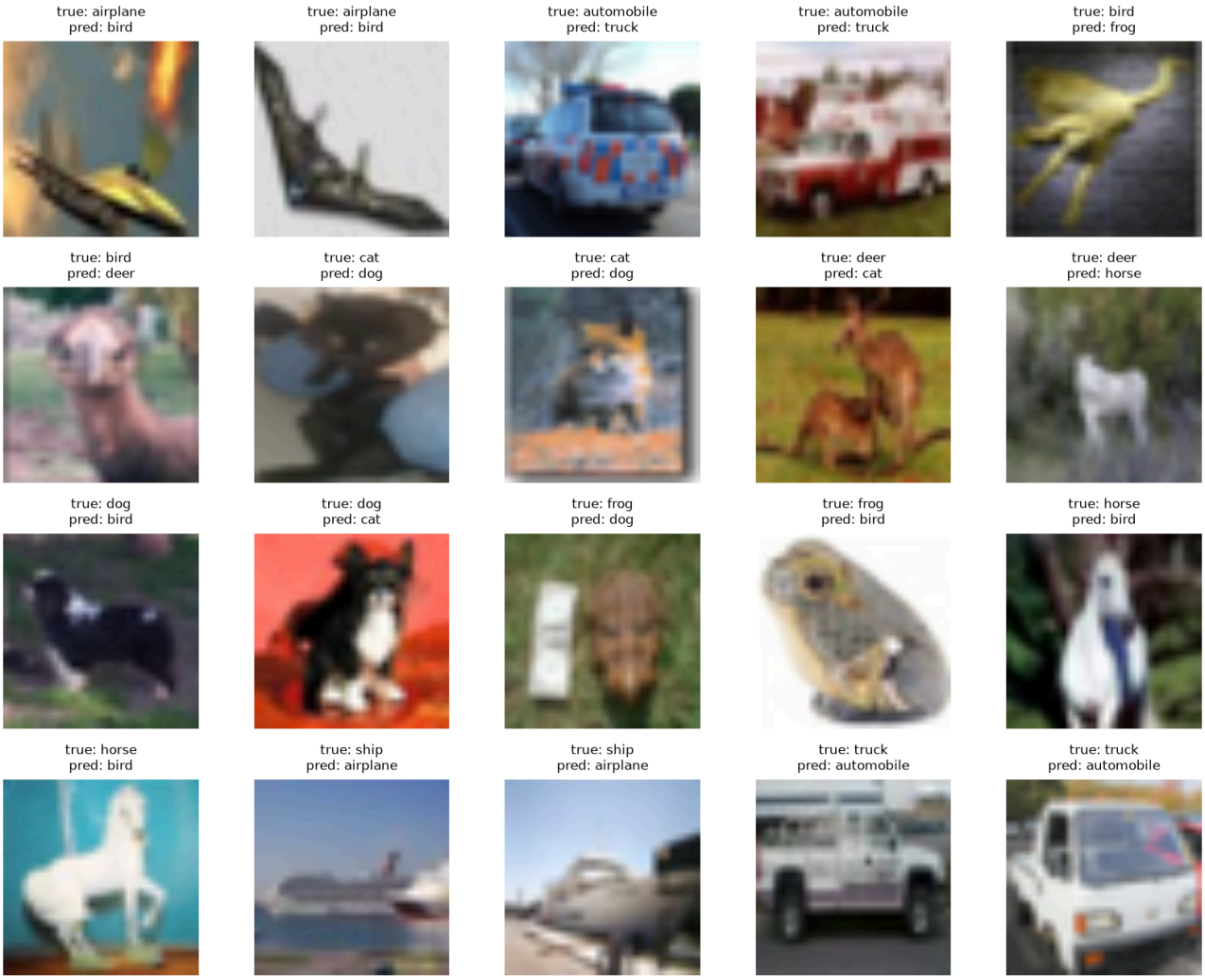
	ResNet-18	ViT full	ViT frozen
Total test errors	23,658	6,047	7,445
automobile<->truck (share of errors)	2,764 (11.7%)	1,506 (24.9%)	1,680 (22.6%)
cat<->dog (share of errors)	2,792 (11.8%)	994 (16.4%)	1,263 (17.0%)
Animal<->vehicle errors (share)	17.0%	10.5%	13.2%

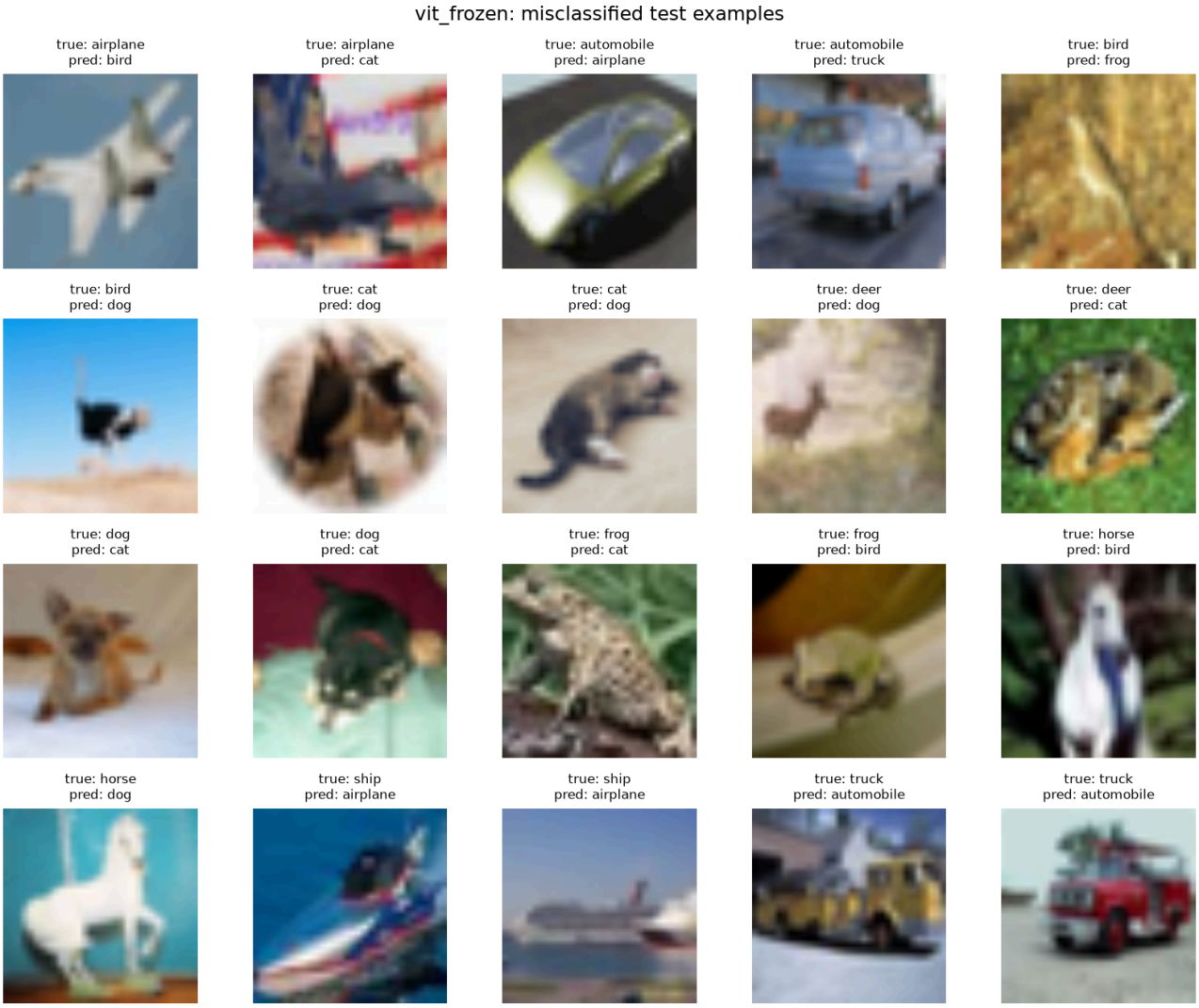
The ViT removes about 75% of the ResNet's errors overall, but only about 45% of its automobile<->truck errors, so that pair makes up a much larger share of what is left (a quarter of all ViT errors). ResNet-specific confusions between animals of similar shape and color, such as dog->deer (973) and deer->horse (900), mostly disappear with pretrained features. What remains are the distinctions that are genuinely hard at 32x32. (pickup trucks and vans versus cars, and cats versus small dogs).

4. Failure cases



vit_full: misclassified test examples





- Images that are hard for every model. The same two horse images are misclassified by all three models. One is a white carousel horse on a blue background (ResNet: ship, ViT full: bird, ViT frozen: dog). The other is a white horse against dark foliage (ResNet: dog, both ViTs: bird). The three models disagree on the wrong answer, which may suggest the image is out of distribution.
- Vehicle class boundaries. ResNet-18 and full ViT both call the same checkered police car a truck. The frozen ViT also labels two fire engines as automobiles, which are clear errors rather than ambiguous cases.
- Background and context cues. ResNet-18 labels a bird silhouetted against plain blue sky as an airplane, and both ViTs label ships photographed against a large sky as airplanes. This may be because sky and water backgrounds can appear similar, especially at a low resolution.

5. Comparison and discussion

1. ResNet vs ViT: scratch-trained ResNet-18 (20 epochs) reaches 73.71% whereas ViT fine-tuned for only 2 epochs reaches 93.28% and wins on every class. The difference comes mainly from pretraining rather than from the architecture itself: the ViT starts from features learned on ImageNet-21k while the ResNet must learn everything from 90,000 low-resolution images. These features transfer to CINIC-10 even though the 32x32 images must be upsampled 7x to the ViT's 224x224 input.
2. Full fine-tune vs frozen head: training only the 7,690-parameter head already yields 91.73% at half the training time. The gain from full fine-tuning is uneven across classes. Interestingly, the frozen-head model is slightly better on airplane, dog, and deer.
3. Compute trade-off: ResNet-18 is far cheaper per epoch: about 3 min per epoch vs about 90 min per epoch for the full ViT fine-tune (224x224 input, 86M params, 196 tokens per image). The ViT needs 10x fewer epochs, but its total training time is still 2.7x longer (179 vs 67 min) on my hardware. The frozen-head ViT halves that time because no backward pass goes through the backbone, but it still needs a full forward pass per image (90 min). Freezing could be made much cheaper by computing the backbone features once and training the head on them.
4. Overfitting behavior: ResNet-18 overfits after about 6 epochs (training loss keeps falling, validation accuracy stops improving), which is expected without augmentation or regularization. The full ViT fine-tune reaches its best validation accuracy after a single epoch and dips slightly in the second. The frozen-head variant is the most stable.
5. Possible improvements: for ResNet-18, weight decay and a learning-rate schedule would likely reduce the overfitting seen after epoch 6. For the ViT we could try a learning rate warm-up and decay, or partially unfreezing only the last few transformer blocks.

6. Variations tried

- ViT frozen-head vs full fine-tune
- ViT full fine-tune learning rate lowered to 5e-5 (from the suggested 1e-3), because a large learning rate on all 86M pretrained weights risks overwriting the pretrained features
- ViT batch size reduced to 16 (from the suggested 32) to fit 224x224 inputs through ViT-Base in the 16 GB of unified memory on the training machine.

Appendix: Source code

src/resnet.py

```
import torch.nn as nn


class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=stride, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(out_channels)

        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_channels),
            )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x):
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + self.shortcut(x)
        return self.relu(out)


class ResNet18(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18, self).__init__()
        self.stem = nn.Sequential(
            nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
        )
        self.stage1 = nn.Sequential(BasicBlock(64, 64), BasicBlock(64, 64))
        self.stage2 = nn.Sequential(BasicBlock(64, 128, stride=2), BasicBlock(128, 128))
        self.stage3 = nn.Sequential(BasicBlock(128, 256, stride=2), BasicBlock(256, 256))
        self.stage4 = nn.Sequential(BasicBlock(256, 512, stride=2), BasicBlock(512, 512))
        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, num_classes)

    def forward(self, x):
        x = self.stem(x)
        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.stage4(x)
        x = self.avgpool(x)
        x = x.flatten(1)
        return self.fc(x)
```

src/data.py

```
from pathlib import Path

from torch.utils.data import DataLoader
from torchvision import datasets, transforms

ROOT = Path(__file__).resolve().parent.parent
CINIC_DIR = ROOT / "CINIC-10"

CINIC_MEAN = (0.47889522, 0.47227842, 0.43047404)
CINIC_STD = (0.24205776, 0.23828046, 0.25874835)
```

```
VIT_SIZE = 224
VIT_MEAN = (0.5, 0.5, 0.5)
VIT_STD = (0.5, 0.5, 0.5)

CLASSES = ("airplane", "automobile", "bird", "cat", "deer",
           "dog", "frog", "horse", "ship", "truck")


def build_transform(for_vit=False, augment=False):
    ops = []
    if augment:
        ops += [transforms.RandomCrop(32, padding=4), transforms.RandomHorizontalFlip()]
    if for_vit:
        ops.append(transforms.Resize(VIT_SIZE))
    mean, std = (VIT_MEAN, VIT_STD) if for_vit else (CINIC_MEAN, CINIC_STD)
    ops += [transforms.ToTensor(), transforms.Normalize(mean, std)]
    return transforms.Compose(ops)


def get_dataset(split, for_vit=False, augment=False):
    transform = build_transform(for_vit=for_vit, augment=augment and split == "train")
    return datasets.ImageFolder(str(CINIC_DIR / split), transform=transform)


def get_loader(split, batch_size=32, for_vit=False, augment=False, num_workers=6):
    dataset = get_dataset(split, for_vit=for_vit, augment=augment)
    return DataLoader(dataset, batch_size=batch_size, shuffle=(split == "train"),
                      num_workers=num_workers, persistent_workers=num_workers > 0)
```

src/engine.py

```
import csv
import time
from pathlib import Path

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import torch
import torch.nn as nn

ROOT = Path(__file__).resolve().parent.parent
CKPT_DIR = ROOT / "outputs" / "checkpoints"
LOG_DIR = ROOT / "outputs" / "logs"
PLOT_DIR = ROOT / "outputs" / "plots"


def get_device():
    if torch.backends.mps.is_available():
        return torch.device("mps")
    if torch.cuda.is_available():
        return torch.device("cuda")
    return torch.device("cpu")


def logits_of(output):
    return output.logits if hasattr(output, "logits") else output


@torch.no_grad()
def compute_accuracy(model, dataloader, device):
    model.eval()
    correct, total = 0, 0
    for images, labels in dataloader:
        images, labels = images.to(device), labels.to(device)
        outputs = logits_of(model(images))
        _, predicted = torch.max(outputs, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)
    return correct / total * 100
```

```
def train(model, train_loader, valid_loader, name, epochs, lr, log_every=100, val_every=1, device=None):
    device = device or get_device()
    model.to(device)
    trainable = [p for p in model.parameters() if p.requires_grad]
    optimizer = torch.optim.Adam(trainable, lr=lr, betas=(0.9, 0.999))
    loss_fn = nn.CrossEntropyLoss()

    loss_rows, val_rows = [], []
    best_acc, step = 0.0, 0
    ckpt_path = CKPT_DIR / f"{name}.pt"
    t0 = time.time()
    print(f"[{name}] device={device} epochs={epochs} lr={lr} "
          f"trainable params={sum(p.numel() for p in trainable):,}")

    for epoch in range(1, epochs + 1):
        model.train()
        running, n_running = 0.0, 0
        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            loss = loss_fn(logits_of(model(images)), labels)
            loss.backward()
            optimizer.step()

            step += 1
            running += loss.item()
            n_running += 1
            if step % log_every == 0:
                avg = running / n_running
                loss_rows.append((step, epoch, avg))
                running, n_running = 0.0, 0
                print(f"[{name}] epoch {epoch} step {step} loss {avg:.4f} ({time.time() - t0:.0f}s)")

        if epoch % val_every == 0 or epoch == epochs:
            acc = compute_accuracy(model, valid_loader, device)
            val_rows.append((step, epoch, acc))
            print(f"[{name}] epoch {epoch} validation accuracy {acc:.2f}%")
            if acc > best_acc:
                best_acc = acc
                torch.save(model.state_dict(), ckpt_path)
                print(f"[{name}] saved best checkpoint -> {ckpt_path}")

    write_csv(LOG_DIR / f"{name}_loss.csv", ("step", "epoch", "loss"), loss_rows)
    write_csv(LOG_DIR / f"{name}_val.csv", ("step", "epoch", "val_acc"), val_rows)
    plot_curves(name, loss_rows, val_rows)
    print(f"[{name}] done in {(time.time() - t0) / 60:.1f} min; best val acc {best_acc:.2f}%")
    return best_acc


def write_csv(path, header, rows):
    with open(path, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(header)
        writer.writerows(rows)


def plot_curves(name, loss_rows, val_rows):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))
    ax1.plot([r[0] for r in loss_rows], [r[2] for r in loss_rows])
    ax1.set_xlabel("step")
    ax1.set_ylabel("training loss")
    ax1.set_title(f"{name}: training loss")
    ax2.plot([r[1] for r in val_rows], [r[2] for r in val_rows], marker="o")
    ax2.set_xlabel("epoch")
    ax2.set_ylabel("validation accuracy (%)")
    ax2.set_title(f"{name}: validation accuracy")
    fig.tight_layout()
    out = PLOT_DIR / f"{name}_curves.png"
    fig.savefig(out, dpi=150)
    print(f"[{name}] curves -> {out}")
```

src/train_resnet.py

```
import argparse

import torch

from data import get_loader
from engine import train
from resnet import ResNet18


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--epochs", type=int, default=20)
    p.add_argument("--batch-size", type=int, default=32)
    p.add_argument("--lr", type=float, default=1e-3)
    p.add_argument("--log-every", type=int, default=100)
    p.add_argument("--val-every", type=int, default=1)
    p.add_argument("--augment", action="store_true")
    p.add_argument("--num-workers", type=int, default=6)
    p.add_argument("--name", default="resnet18")
    args = p.parse_args()

    torch.manual_seed(0)
    model = ResNet18(num_classes=10)
    train_loader = get_loader("train", batch_size=args.batch_size, augment=args.augment,
                              num_workers=args.num_workers)
    valid_loader = get_loader("valid", batch_size=256, num_workers=args.num_workers)
    train(model, train_loader, valid_loader, name=args.name, epochs=args.epochs, lr=args.lr,
          log_every=args.log_every, val_every=args.val_every)


if __name__ == "__main__":
    main()
```

src/train_vit.py

```
import argparse

import torch
from transformers import ViTForImageClassification

from data import get_loader
from engine import train

MODEL_ID = "google/vit-base-patch16-224-in21k"


def build_vit(freeze=False):
    model = ViTForImageClassification.from_pretrained(MODEL_ID, num_labels=10)
    if freeze:
        for param in model.vit.parameters():
            param.requires_grad = False
    return model


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--epochs", type=int, default=2)
    p.add_argument("--batch-size", type=int, default=16)
    p.add_argument("--lr", type=float, default=None)
    p.add_argument("--freeze", action="store_true")
    p.add_argument("--log-every", type=int, default=100)
    p.add_argument("--num-workers", type=int, default=6)
    args = p.parse_args()

    lr = args.lr if args.lr is not None else (1e-3 if args.freeze else 5e-5)
    name = "vit_frozen" if args.freeze else "vit_full"

    torch.manual_seed(0)
    model = build_vit(freeze=args.freeze)
    train_loader = get_loader("train", batch_size=args.batch_size, for_vit=True,
                              num_workers=args.num_workers)
    valid_loader = get_loader("valid", batch_size=64, for_vit=True, num_workers=args.num_workers)
```

```
    train(model, train_loader, valid_loader, name=name, epochs=args.epochs, lr=lr,
          log_every=args.log_every, val_every=1)


if __name__ == "__main__":
    main()
```

src/evaluate.py

```
import argparse

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import torch
from sklearn.metrics import confusion_matrix

from data import CLASSES, CINIC_MEAN, CINIC_STD, VIT_MEAN, VIT_STD, get_loader
from engine import CKPT_DIR, PLOT_DIR, get_device, logits_of
from resnet import ResNet18


@torch.no_grad()
def collect_predictions(model, loader, device, per_class=2):
    model.eval()
    y_true, y_pred = [], []
    failures = {c: [] for c in range(len(CLASSES))}
    for images, labels in loader:
        outputs = logits_of(model(images.to(device)))
        predicted = outputs.argmax(1).cpu()
        y_true.append(labels)
        y_pred.append(predicted)
        for i in (predicted != labels).nonzero().flatten():
            true = labels[i].item()
            if len(failures[true]) < per_class:
                failures[true].append((images[i], true, predicted[i].item()))
    failures = [f for c in range(len(CLASSES)) for f in failures[c]]
    return torch.cat(y_true).numpy(), torch.cat(y_pred).numpy(), failures


def plot_confusion(cm, name):
    fig, ax = plt.subplots(figsize=(8, 7))
    im = ax.imshow(cm, cmap="Blues")
    ax.set_xticks(range(10), CLASSES, rotation=45, ha="right")
    ax.set_yticks(range(10), CLASSES)
    ax.set_xlabel("predicted")
    ax.set_ylabel("true")
    ax.set_title(f"{name}: confusion matrix (test set)")
    for i in range(10):
        for j in range(10):
            color = "white" if cm[i, j] > cm.max() / 2 else "black"
            ax.text(j, i, cm[i, j], ha="center", va="center", fontsize=7, color=color)
    fig.colorbar(im)
    fig.tight_layout()
    out = PLOT_DIR / f"{name}_confusion.png"
    fig.savefig(out, dpi=150)
    print(f"confusion matrix -> {out}")


def plot_failures(failures, name, mean, std):
    mean = torch.tensor(mean).view(3, 1, 1)
    std = torch.tensor(std).view(3, 1, 1)
    fig, axes = plt.subplots(4, 5, figsize=(11, 9))
    for ax in axes.flat:
        ax.axis("off")
    for ax, (img, true, pred) in zip(axes.flat, failures):
        ax.imshow((img * std + mean).clamp(0, 1).permute(1, 2, 0))
        ax.set_title(f"true: {CLASSES[true]}\npred: {CLASSES[pred]}", fontsize=8)
    fig.suptitle(f"{name}: misclassified test examples")
    fig.tight_layout()
    out = PLOT_DIR / f"{name}_failures.png"
```

```
    fig.savefig(out, dpi=150)
    print(f"failure examples -> {out}")


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--model", choices=("resnet", "vit"), required=True)
    p.add_argument("--name", default=None)
    p.add_argument("--freeze", action="store_true")
    p.add_argument("--batch-size", type=int, default=128)
    p.add_argument("--num-workers", type=int, default=6)
    args = p.parse_args()

    name = args.name or ("resnet18" if args.model == "resnet" else "vit_full")
    device = get_device()

    if args.model == "resnet":
        model = ResNet18(num_classes=10)
        mean, std = CINIC_MEAN, CINIC_STD
    else:
        from train_vit import build_vit
        model = build_vit(freeze=args.freeze)
        mean, std = VIT_MEAN, VIT_STD

    state = torch.load(CKPT_DIR / f"{name}.pt", map_location=device)
    model.load_state_dict(state)
    model.to(device)

    loader = get_loader("test", batch_size=args.batch_size, for_vit=(args.model == "vit"),
                        num_workers=args.num_workers)
    y_true, y_pred, failures = collect_predictions(model, loader, device)

    acc = (y_true == y_pred).mean() * 100
    print(f"\n{name} test accuracy: {acc:.2f}%\n")
    cm = confusion_matrix(y_true, y_pred)
    per_class = cm.diagonal() / cm.sum(axis=1) * 100
    for cls, a in zip(CLASSES, per_class):
        print(f"  {cls:<12} {a:6.2f}%")

    np.savetxt(PLOT_DIR / f"{name}_confusion.csv", cm, fmt="%d", delimiter=",")
    plot_confusion(cm, name)
    plot_failures(failures, name, mean, std)


if __name__ == "__main__":
    main()
```

run_queue.sh

```
#!/bin/bash
cd "$(dirname "$0")/src" || exit 1
PY=../.venv/bin/python

echo "[queue] waiting for train_resnet.py to finish ($(date))"
while pgrep -f "train_resnet.py" > /dev/null; do sleep 120; done
echo "[queue] ResNet training done ($(date))"

$PY -u train_vit.py            2>&1 | tee ../outputs/logs/vit_full_train.log
$PY -u train_vit.py --freeze   2>&1 | tee ../outputs/logs/vit_frozen_train.log

$PY -u evaluate.py --model resnet                     2>&1 | tee ../outputs/logs/resnet18_eval.log
$PY -u evaluate.py --model vit --name vit_full        2>&1 | tee ../outputs/logs/vit_full_eval.log
$PY -u evaluate.py --model vit --name vit_frozen --freeze 2>&1 | tee ../outputs/logs/vit_frozen_eval.log

echo "[queue] all runs complete ($(date))"
```